Guía del Proyecto N°1 (Control y supervisión de sistema secuencial electroneumático). Explica cómo descargar el proyecto desde GitHub en otra computadora, instalarlo y ejecutarlo con los archivos `.bat` *del repositorio, dónde está cada uno y por qué se eligió ese formato. Los requisitos de Python, Qt y OPC DA están en la Guía 1 (*`guia_instalacion_python_qt_opc.pdf`*), en esta misma carpeta.*

Índice

I. Qué se descarga y de dónde **1**

II. Requisitos previos **2**

III. Descargar el proyecto **2**

III.1. Con Git (recomendado) . . . 3
III.2. Sin Git (archivo ZIP) . . . 3
III.3. Actualizar . . . 3
III.4. Qué contiene la carpeta . . . 4

IV. Por qué archivos .bat **4**

IV.1. Anatomía de un lanzador . . . 5
IV.2. Límites y cuidados . . . 5

V. Dónde está cada .bat **6**

VI. Instalar todo con instalar.bat **7**

VII. Ejecutar el sistema **8**

VII.1. Servidor del gemelo digital (opcional, otra PC) . . . 8

VIII. Problemas típicos **9**

IX. Pruebas de aceptación de la réplica **9**

I. Qué se descarga y de dónde

El código está en dos repositorios de GitHub (Tabla 1). Los dos contienen el mismo gateway, HMI, gemelo digital, pruebas y lanzadores `.bat`: SCADA-Digital-Twin es el repositorio de desarrollo del HMI y Proyecto1-SCADA es el entregable del proyecto, donde el código está en `01_Integracion\04_Software` junto con el programa del PLC, la simulación y estas guías.

Repositorio	Contenido	Cuándo se descarga
SCADA-Digital-Twin [1]	Gateway OPC, HMI (PySide6 + QML), gemelo digital, puente, pruebas, lanzadores de `lab\` y de `digital_twin\despliegue\`	Siempre: es lo que se instala en la PC del laboratorio.
Proyecto1-SCADA [2]	Entregable del proyecto: el mismo código en `01_Integracion\04_Software`, el programa del PLC (`PROYECTO1_TEST.RSS`), el modelo de Simulink, estas guías y `instalacion_python\gbda_aut.dll`	Para tener todo el proyecto en un solo lugar, o para el programa del PLC y la DLL de OpenOPC (Guía 1, §IV).

Tabla 1: Repositorios del proyecto.

Si se usa Proyecto1-SCADA, todos los comandos de esta guía se ejecutan desde `01_Integracion\04_Software` en lugar de `C:\Proyectos\HMI`: los lanzadores calculan sus rutas a partir de su propia ubicación. Lo que se descarga de GitHub es lo último que se subió con `git push`: los cambios que sólo existen en la PC de desarrollo no llegan al laboratorio. Antes de ir al laboratorio, comprueba que la versión que se probó está en GitHub.

La Figura 1 resume el camino completo, desde una PC sin nada hasta el HMI conectado al PLC. Cada paso tiene un lanzador o una comprobación.

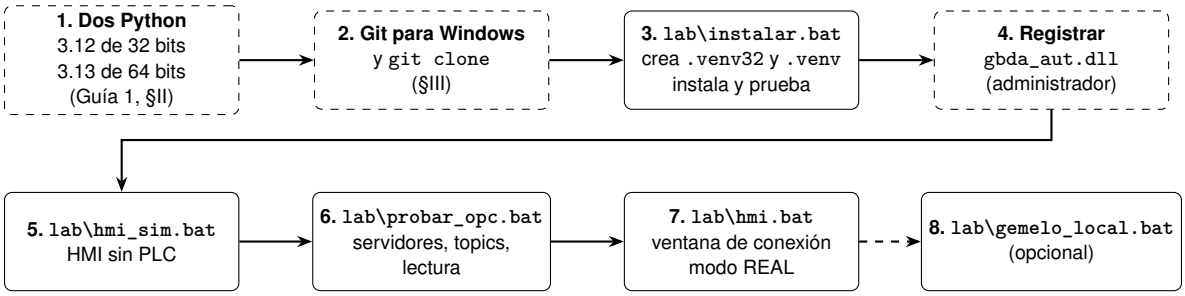


Figura 1: Pasos para replicar el proyecto. Los recuadros continuos son lanzadores `.bat`; los punteados son pasos manuales.

II. Requisitos previos

Requisito	Detalle	Se comprueba con
Python 3.12 de 32 bits y 3.13 de 64 bits	Con el *py launcher* (Guía 1, §II)	`py -0p`
Git para Windows	Opcional pero recomendado: permite actualizar con un comando [3]	`git --version`
Internet	Para `git clone` y para que `instalar.bat` descargue los paquetes. Sin internet: ruedas (Guía 1, §II.6)	—
RSLinx Classic con OPC	Sólo para el modo REAL (Guía 1, §IV)	*OPC Test Client*
Cuenta de administrador	Sólo para registrar `gbda_aut.dll`	—

Tabla 2: Requisitos para replicar el proyecto.

III. Descargar el proyecto

Usa una carpeta corta, sin espacios ni acentos, por ejemplo `C:\Proyectos\HMI`. Los lanzadores funcionan aunque la ruta tenga espacios, pero los comandos se escriben más fácil sin ellos.

DQ DQ P1 P1

III.1. Con Git (recomendado)

1. Instala Git para Windows [3] con las opciones por defecto. La opción por defecto *Checkout Windows-style, commit Unix-style line endings* es importante: deja los `.bat` con fin de línea CRLF (§IV.2).
2. Abre una consola (Símbolo del sistema o PowerShell) y clona el repositorio [4]:

Listado 1: Descarga del repositorio del HMI.

```
mkdir C:\Proyectos
cd C:\Proyectos
git clone https://github.com/DaanMaldonadoU/SCADA-Digital-Twin.git HMI
cd HMI
rem Alternativa: el proyecto completo (codigo, programa del PLC, guias y DLL)
git clone https://github.com/DaanMaldonadoU/Proyecto1-SCADA.git C:\Proyectos\P1
cd /d C:\Proyectos\P1\01_Integracion\04_Software
```

Si el repositorio es privado, Git abre el navegador para iniciar sesión en GitHub la primera vez (Git Credential Manager guarda la sesión). Si pide una contraseña en texto, GitHub exige un *token* personal en su lugar.

III.2. Sin Git (archivo ZIP)

En la página del repositorio: *Code* → *Download ZIP*. Descomprime (clic derecho → *Extraer todo*), renombra la carpeta `SCADA-Digital-Twin-main` a `HMI` y muévela a `C:\Proyectos\HMI`. Para actualizar hay que repetir la descarga y copiar encima, conservando la carpeta `datos\`. El ZIP puede traer los `.bat` con fin de línea LF; si un lanzador se comporta de forma extraña, usa `git clone` (§IV.2).

III.3. Actualizar

Listado 2: Traer la última versión publicada.

```
cd /d C:\Proyectos\HMI
git pull
rem Si dice que hay cambios locales:  git stash  ->  git pull  ->  git stash pop
```

Después de actualizar, vuelve a ejecutar `lab\instalar.bat` si cambió algún `requirements-*.txt`; no borra nada y sólo instala lo que falte.

III.4. Qué contiene la carpeta

```
HMI/ ..... raíz del repositorio SCADA-Digital-Twin
├── lab/ ..... lanzadores para la PC del laboratorio (§V)
│   ├── instalar.bat, probar_opc.bat, gateway.bat, hmi.bat
│   └── hmi_sim.bat, gemelo_local.bat, pruebas.bat
├── gateway/ ..... gateway OPC / simulador y diagnóstico OPC
├── ui/ ..... HMI: Python + QML; ui/qml/HMI.qmlproject
├── core/ ..... tabla de variables, protocolo, calibración, alarmas
├── digital_twin/ ..... gemelo digital web
│   └── despliegue/ ..... lanzadores de la PC servidor del gemelo
├── puente/ ..... enlace del HMI con el servidor del gemelo
├── tests/ ..... pruebas automáticas (pytest)
├── docs/, mds/ ..... guía de instalación y documentación técnica
├── requirements-gateway.txt ..... paquetes de .venv32
├── requirements-hmi.txt ..... paquetes de .venv
├── .venv32/, .venv/ ..... los crea instalar.bat; no están en Git
└── datos/ ..... se crea al ejecutar: conexión, calibración, historial
```

Figura 2: Carpetas del repositorio del HMI después de instalar.

IV. Por qué archivos .bat

Ejecutar el sistema a mano exige recordar, en cada arranque, tres cosas que fallan con facilidad en el laboratorio: (1) qué intérprete usa cada proceso (el gateway sólo funciona con el Python de 32 bits); (2) estar en la raíz del repositorio, porque `python -m gateway` busca los paquetes ahí; y (3) argumentos largos como `--servidor "RSLinx OPC Server" --topic PROYECTO1_TEST --periodo 100`. Un archivo `.bat` es un guion de `cmd.exe` que guarda esos comandos [5]. El proyecto los usa por estas razones:

1. **Fijan el intérprete correcto.** Cada lanzador llama al `python.exe` del entorno que le toca (`.venv32\Scripts\python.exe` o `.venv\Scripts\python.exe`). El error más común, abrir el gateway con el Python de 64 bits y obtener *Class not registered*, no puede ocurrir.
2. **Funcionan con doble clic y desde cualquier carpeta.** La línea `cd /d "%~dp0.."` cambia a la raíz del repositorio a partir de la ubicación del propio `.bat` (`%~dp0` es la unidad y la carpeta del archivo [6]), aunque la ruta tenga espacios o esté en otra unidad.
3. **No requieren instalar nada ni cambiar permisos.** `cmd.exe` existe en cualquier Windows. Un guion de PowerShell (`.ps1`), en cambio, está bloqueado por defecto en Windows 10 y 11 (política de ejecución *Restricted*) [7], y cambiar esa política en una PC del laboratorio no siempre está permitido.
4. **Guardan los valores del laboratorio, pero aceptan otros.** `gateway.bat` arranca en REAL sobre `PROYECTO1_TEST` con periodo de 100 ms si no recibe argumentos (`%1`, `%2`); los demás pasan todo lo que reciben con `%*`.
5. **El error queda a la vista.** Terminan con `pause`: si algo falla, la ventana no se cierra y el mensaje se puede leer y copiar a la bitácora.
6. **Comprueban antes de actuar y explican qué falta.** `instalar.bat` verifica que exista `py -3.12-32` y, si no, dice qué instalador descargar; tampoco recrea un entorno que ya existe.

7. **Se documentan solos y viajan con el código.** Cada uno empieza con líneas `rem Uso:` y ejemplos; como están en Git, quien clona el repositorio obtiene exactamente los mismos pasos que se probaron. La instalación se vuelve reproducible.

IV.1. Anatomía de un lanzador

Listado 3: `lab\hmi.bat` completo.

```
@echo off
rem HMI PySide6 (Python de 64 bits) contra el gateway que ya corre en 5555/5556.
rem Uso: hmi.bat [argumentos extra de ui\app.py]   p. ej.  hmi.bat --pantalla-completa
cd /d "%~dp0.."
.venv\Scripts\python.exe ui\app.py %*
pause
```

Línea	Qué hace
`@echo off`	No repite cada comando en la consola; sólo se ven los mensajes del programa.
`rem ...`	Comentario: qué hace el lanzador y cómo se usa.
`cd /d "%~dp0.."`	Va a la carpeta padre de `lab\`, es decir, a la raíz del repositorio. `/d` cambia también de unidad.
`.venv\Scripts\python.exe ui\app.py %*`	Ejecuta el HMI con el Python de 64 bits del entorno y le pasa todos los argumentos.
`pause`	Espera una tecla antes de cerrar la ventana.

Tabla 3: Partes de `hmi.bat`.

Listado 4: `lab\gateway.bat` (extracto): valores por defecto y argumentos.

```
set MODO=%1
if "%MODO%"=="" set MODO=REAL
set TOPIC=%2
if "%TOPIC%"=="" set TOPIC=PROYECTO1_TEST
cd /d "%~dp0.."
if /i "%MODO%"=="SIM" (
  .venv32\Scripts\python.exe -m gateway --modo SIM %3 %4 %5 %6 %7 %8
) else (
  .venv32\Scripts\python.exe -m gateway --modo %MODO% --servidor "RSLinx Local OPC Server" --topic %TOPIC%
    --periodo 100 %3 %4 %5 %6 %7 %8
)
```

IV.2. Límites y cuidados

- **Sólo Windows.** Es suficiente: RSLinx y OPC DA sólo existen en Windows.
- **Fin de línea CRLF.** Con fin de línea LF, `cmd` puede saltarse etiquetas de `goto` y `call :etiqueta` (se observó en `Informe\compilar_documentos.bat`). En Proyecto1-SCADA lo fija `.gitattributes` (`*.bat text eol=crlf`); en SCADA-Digital-Twin depende de que Git convierta al descargar (opción por defecto de Git para Windows).
- **Sin acentos en los mensajes.** La consola lee el `.bat` con la página de códigos del sistema (850 en Windows en español), no en UTF-8; por eso los `echo` se escriben sin acentos («instalacion»).

- **Variables dentro de bloques.** `cmd` expande `%VAR%` al leer un bloque `( ... )` completo, antes de ejecutarlo; una variable leída dentro del bloque no se ve ahí. Por eso `instalar_servidor.bat` usa `goto` en lugar de bloques.

V. Dónde está cada .bat

La Tabla 4 reúne los lanzadores de los dos repositorios.

Archivo	Qué hace	Intérprete	Ejemplo
SCADA-Digital-Twin, carpeta `lab\` (PC del laboratorio)			
`instalar.bat`	Crea `.venv32` y `.venv`, instala los paquetes, comprueba las importaciones y corre las pruebas. Una vez por PC	`py -3.12-32`, `py -3.13-64`	`lab\instalar.bat`
`hmi_sim.bat`	HMI con gateway simulado: la primera prueba, sin PLC ni RSLinx	`.venv`	`lab\hmi_sim.bat`
`probar_opc.bat`	Diagnóstico OPC DA: servidores, topics, items, tiempos, escritura de preajustes	`.venv32`	`lab\probar_opc.bat --servidores`
`hmi.bat`	HMI; si no hay gateway abre la ventana de conexión, que lo arranca	`.venv`	`lab\hmi.bat --pantalla-completa`
`gateway.bat`	Gateway solo (REAL, `PROYECTO1_TEST`, 100 ms por defecto), para usarlo con dos consolas	`.venv32`	`lab\gateway.bat SIM`
`gemelo_local.bat`	Gemelo digital en la misma PC, sin internet, en `http://127.0.0.1:3002`	`.venv`	`lab\gemelo_local.bat --gateway SIM`
`pruebas.bat`	Pruebas automáticas (pytest), sin PLC	`.venv`	`lab\pruebas.bat`
SCADA-Digital-Twin, carpeta `digital_twin\despliegue\` (PC servidor del gemelo)			
`instalar_servidor.bat`	Crea `.venv-web`, instala aiohttp y paho-mqtt, genera la clave del puente y la primera cuenta admin	`py -3.13-64`	doble clic
`servidor.bat`	Servidor web del gemelo en el puerto 3002	`.venv-web`	`servidor.bat --publico`
`cuentas.bat`	Alta, rol, contraseña y baja de cuentas; auditoría	`.venv-web`	`cuentas.bat crear ana operador`
Proyecto1-SCADA (documentación)			
`Informe\compilar.bat`	Compila sólo el informe (`main.pdf`)	— (MiKTeX)	doble clic
`Informe\compilar_documentos.bat`	Regenera Gantt, tablas y costos; compila el informe y los 18 documentos de carpeta	`py -I` y MiKTeX	doble clic
`instalacion_python\compilar_guias.bat`	Compila estas dos guías con el formato del informe	— (MiKTeX)	doble clic

Tabla 4: Lanzadores `.bat` del proyecto.

Todos los lanzadores de `lab\` y de `despliegue\` empiezan por cambiar a la raíz del repositorio, así que da igual desde dónde se ejecuten. Cada uno tiene un equivalente sin `.bat`, útil para entender qué hace o para depurar:

Listado 5: Comandos equivalentes, desde la raíz del repositorio.

```
.venv32\Scripts\python.exe -m gateway.diagnostico_opc --servidores
.venv32\Scripts\python.exe -m gateway --modo REAL --servidor "RSLinx OPC Server" --topic PROYECTO1_TEST
    --periodo 100 --fuente cache
.venv\Scripts\python.exe ui\app.py
.venv\Scripts\python.exe ui\app.py --lanzar-gateway SIM
```

```
.venv\Scripts\python.exe -m digital_twin.local --gateway SIM
.venv\Scripts\python.exe -m pytest tests -q
```

VI. Instalar todo con instalar.bat

Con los dos Python instalados (Guía 1, §II), haz doble clic en `lab\instalar.bat`. Tarda unos minutos con internet y avanza en cuatro pasos (Figura 3). Si un paso falla, se detiene con un mensaje y la ventana queda abierta.

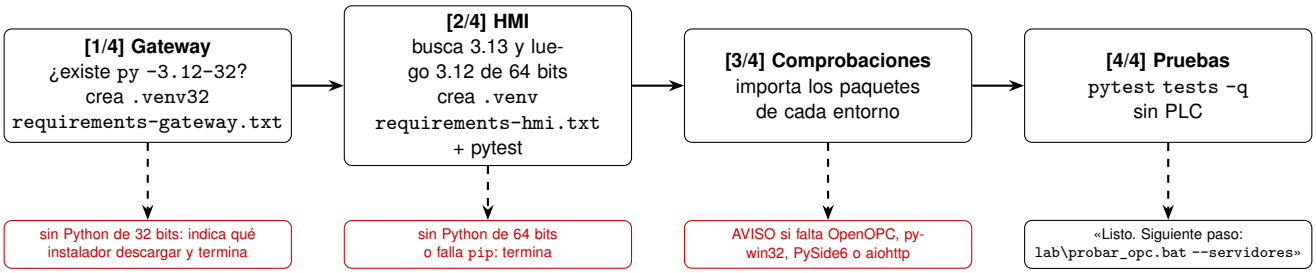


Figura 3: Lo que hace `lab\instalar.bat`.

Al terminar, la consola debe mostrar, en este orden: la versión de cada Python, `gateway 32 bits: dependencias OK`, `HMI 64 bits: dependencias OK`, el resumen de pytest con todas las pruebas `passed` (en la PC de desarrollo, el 9 de octubre de 2026: `212 passed` en unos 2,5 min) y la línea «Listo. Siguiente paso: `lab\probar_opc.bat --servidores`». Si una prueba sale `skipped` por Playwright, es normal: es una prueba opcional del gemelo.

El Listado 6 muestra cómo trabaja el primer paso.

Listado 6: `lab\instalar.bat`, paso 1 de 4 (extracto).

```
setlocal
cd /d "%~dp0.."
echo === [1/4] Python de 32 bits para el gateway ===
py -3.12-32 -c "import sys; print('Python', sys.version.split()[0], '32 bits' if sys.maxsize < 2**32 else '64
     bits')" 2>nul
if errorlevel 1 (
  echo   NO se encontro Python 3.12 de 32 bits. Instala python-3.12.x.exe ^(Windows installer 32-bit^)
  echo   desde python.org marcando "py launcher" y vuelve a ejecutar este archivo.
  goto :fin
)
if not exist .venv32\Scripts\python.exe py -3.12-32 -m venv .venv32
.venv32\Scripts\python.exe -m pip install --upgrade pip
.venv32\Scripts\python.exe -m pip install -r requirements-gateway.txt
if errorlevel 1 ( echo   Fallo la instalacion del gateway. & goto :fin )
```

Ejecutarlo otra vez es seguro: no recrea los entornos que ya existen (`if not exist ...`) y `pip` sólo instala lo que falte. `instalar.bat` **no** hace tres cosas, que van aparte:

1. Instalar Python ni Git (§III y Guía 1, §II).
2. Registrar `gbda_aut.dll`, porque requiere administrador (Guía 1, §IV.4). La DLL está en la carpeta `instalacion_python` del repositorio Proyecto1-SCADA; si no la tienes, se obtiene de la carpeta `lib/` de OpenOPC [8].
3. Configurar RSLinx (driver y topic; Guía 1, §IV.6).

VII. Ejecutar el sistema

Paso	Lanzador	Cuándo	Qué debes ver
1	`lab\hmi_sim.bat`	Primera prueba, sin PLC	Chip SIM, `Good 24/24`, GRAFCET en X0; START avanza X1…X7
2	`lab\pruebas.bat`	Tras instalar o actualizar	Todas `passed`
3	`lab\probar_opc.bat --servidores` y `--topics`	Con RSLinx abierto	El servidor de RSLinx y `PROYECTO1_TEST`
4	`lab\probar_opc.bat --topic PROYECTO1_TEST --modo REAL --n 30`	Con el SLC en RUN	Todo `Good`; caché de 6 a 8 ms
5	`lab\hmi.bat`	Operación con el PLC	Ventana de conexión: REAL, *Arrancar aquí*, *Conectar*; pie «CONECTADO · modo REAL»
6	`lab\gemelo_local.bat`	Opcional, con el gateway corriendo	El navegador abre `http://127.0.0.1:3002` con la planta en línea

Tabla 5: Orden de uso de los lanzadores.

La alternativa al paso 5 son dos consolas: `lab\gateway.bat` en una (REAL sobre `PROYECTO1_TEST`) y `lab\hmi.bat` en otra. En ese caso, en la ventana de conexión se elige *Usar uno existente*; si se pulsa *Arrancar aquí*, el segundo gateway no puede abrir los puertos 5555/5556 y termina con error. Para cerrar todo basta con cerrar la ventana del HMI o pulsar *Desconectar*.

VII.1. Servidor del gemelo digital (opcional, otra PC)

El gemelo publicado en internet corre en una PC servidor distinta de la del laboratorio. Ahí se usan los tres lanzadores de `digital_twin\despliegue\`, con su propio entorno `.venv-web`:

Listado 7: Lanzadores de la PC servidor del gemelo.

```
digital_twin\despliegue\instalar_servidor.bat
digital_twin\despliegue\servidor.bat
digital_twin\despliegue\cuentas.bat
digital_twin\despliegue\cuentas.bat crear ana operador
```

La publicación (proxy inverso o túnel) se describe en `mds/DESPLIEGUE_SERVIDOR.md` del repositorio del HMI. No es necesaria para operar la planta.

VIII. Problemas típicos

Síntoma	Causa probable y solución
`git` no se reconoce	Falta Git para Windows o la consola se abrió antes de instalarlo. Instalar y abrir una consola nueva.
`git clone` pide usuario y contraseña	Repositorio privado. Iniciar sesión en el navegador o usar un *token* personal.
La ventana del `.bat` se abre y se cierra	Se ejecutó un comando que no termina en `pause`, o el antivirus lo bloqueó. Abrir una consola y ejecutarlo desde ahí para ver el mensaje.
«NO se encontro Python 3.12 de 32 bits»	Falta ese Python o el *py launcher* (Guía 1, §II).
«El sistema no puede encontrar la ruta especificada» con `.venv32\Scripts\python.exe`	No se ha ejecutado `instalar.bat`, o falló el paso [1/4].
Un lanzador ignora un `goto` o se comporta de forma extraña	Fin de línea LF (descarga en ZIP). Volver a descargar con `git clone`.
`git pull` dice que hay cambios locales	`git stash`, `git pull`, `git stash pop`.
Puertos 5555/5556 ocupados	Ya hay un gateway corriendo (otra consola o un HMI anterior). Cerrarlo o usar *Usar uno existente*.

Tabla 6: Problemas frecuentes al replicar el proyecto.

IX. Pruebas de aceptación de la réplica

ID	Prueba	Comando o acción	Resultado esperado	✔
R-01	Git	`git --version`	Muestra la versión	☐
R-02	Descarga	`git clone ... HMI`	Existe `C:\Proyectos\HMI\lab\instalar.bat`	☐
R-03	Fin de línea	`git ls-files --eol lab/*.bat`	Columna `w/crlf` en los siete	☐
R-04	Instalación	`lab\instalar.bat`	Las dos líneas `dependencias OK` y pytest sin `failed`	☐
R-05	Entornos	`dir .venv32 .venv`	Existen las dos carpetas	☐
R-06	Sin PLC	`lab\hmi_sim.bat`	Chip SIM, `Good 24/24`	☐
R-07	DLL	Guía 1, pruebas P-07 a P-09	Clase `Graybox.OPC.DAWrapper` creada	☐
R-08	OPC	`lab\probar_opc.bat --servidores`	Lista el servidor de RSLinx	☐
R-09	Con el PLC	`lab\hmi.bat`, modo REAL	«CONECTADO · modo REAL», `Good 24/24`	☐
R-10	Actualización	`git pull`	*Already up to date* o lista de cambios, sin errores	☐

Tabla 7: Pruebas para dar por replicado el proyecto en otra PC.

Referencias

[1] D. Y. Maldonado Uriostigue, *SCADA-Digital-Twin* (HMI, gateway y gemelo digital), `https://github.com/DaanMaldonadoU/SCADA-Digital-Twin`.

DQ DQ P1 P1

[2] D. Y. Maldonado Uriostigue, *Proyecto1-SCADA*, `https://github.com/DaanMaldonadoU/Proyecto1-SCADA`.

[3] Git, *Git for Windows*, `https://git-scm.com/download/win`.

[4] GitHub, *Cloning a repository*, `https://docs.github.com/repositories/creating-and-managing-repositories/cloning-a-repository`.

[5] Microsoft, *Windows commands* (referencia de `cmd`), `https://learn.microsoft.com/windows-server/administration/windows-commands/windows-commands`.

[6] Microsoft, *call* (parámetros de lotes y modificadores como `%~dp0`), `https://learn.microsoft.com/windows-server/administration/windows-commands/call`.

[7] Microsoft, *about_Execution_Policies*, `https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about_execution_policies`.

[8] M. Kwiatkowski, *openopc* (bifurcación de OpenOPC for Python de B. Barnreiter), repositorio con `lib/gbda_aut.dll`, `https://github.com/mkwiatkowski/openopc`.